Theory Assignment - 3

JavaScript

1. What is the difference between frontend and backend JavaScript?

Frontend JavaScript runs in the user's browser and is used to make web pages interactive, such as validating forms, changing themes, and responding to clicks. Backend JavaScript runs on a server, commonly using Node.js, and handles tasks such as processing requests, working with databases, and authentication.

Example: JavaScript that checks a signup form in the browser is frontend JavaScript. JavaScript that saves the signup details to a database on a server is backend JavaScript.

2. What is DOM?

DOM stands for Document Object Model. It is a programming representation of an HTML document as a tree of objects. JavaScript can use the DOM to read or change page content, attributes, styles, and elements.

Example:

```
document.getElementById("output").innerHTML = "Hello";
```

This finds the element with the id "output" and changes its content to Hello.

3. What is an event in JavaScript?

An event is an action or occurrence that JavaScript can detect and respond to. Events can be caused by the user or by the browser, such as clicking a button, typing in a field, submitting a form, or loading a page.

Example:

```
<button onclick="alert('Button clicked')">Click</button>
```

The click event runs the alert when the button is clicked.

4. What is event loop in JavaScript?

The event loop is the mechanism that helps JavaScript handle asynchronous tasks while running its main code on a call stack. When the call stack is free, the event loop allows queued callbacks to run. Promise callbacks (microtasks) are processed before ordinary task-queue callbacks such as setTimeout callbacks.

Example:

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
```

Output:

```
A
C
B
```

The timer callback runs after the current synchronous code finishes, even when its delay is zero.

5. What is hoisting in JavaScript?

Hoisting describes how JavaScript processes declarations before executing code in their scope. Function declarations can generally be called before their position in the source code. Variables declared with var are initialized as undefined before their assignment, while let and const are unavailable before their declaration is reached because they are in the temporal dead zone.

Example:

```
console.log(x); // undefined
var x = 10;
```

The declaration of x is processed before execution, but its assignment happens at the original line.

6. What is event bubbling?

Event bubbling is the process in which an event triggered on a nested element travels upward through its parent elements. This allows a parent element to handle events that occur on its children. It can be stopped with event.stopPropagation().

Example: If a button is inside a div and the button is clicked, the button's click handler runs and the event can then bubble to the div's click handler.

7. What is the difference between named function, anonymous function and arrow function?

A named function has a name and can be called using that name. An anonymous function has no function name of its own and is often assigned to a variable or passed as a callback. An arrow function uses the => syntax and has lexical this, meaning it takes this from its surrounding scope.

Examples:

```
function add(a, b) {
   return a + b;
}

const multiply = function(a, b) {
   return a * b;
};

const subtract = (a, b) => a - b;
```

Named functions can be convenient for reuse and debugging. Anonymous and arrow functions are commonly used as callbacks and short function expressions. Arrow functions are not interchangeable with regular functions in every situation, especially when this or a constructor is involved.

8. Explain for-of and for-in loop?

A for...of loop iterates over values in an iterable, such as an array or string. A for...in loop iterates over enumerable property keys of an object. For arrays, for...of is usually preferred when the goal is to process each value.

Example:

```
let colors = ["red", "blue"];

for (let color of colors) {
   console.log(color); // red, blue
}

let person = { name: "Asha", age: 20 };
```

```
for (let key in person) {
    console.log(key, person[key]); // name Asha, age 20
}
```

9. Explain any five array methods?

1. push(): Adds one or more elements to the end of an array.
   Example: numbers.push(4);
2. pop(): Removes and returns the last element.
   Example: let last = numbers.pop();
3. shift(): Removes and returns the first element.
   Example: let first = numbers.shift();
4. unshift(): Adds one or more elements to the beginning.
   Example: numbers.unshift(0);
5. map(): Creates a new array by applying a function to each element.
   Example: let doubled = numbers.map(n => n * 2);

push(), pop(), shift(), and unshift() modify the original array. map() returns a new array and does not modify the original array by itself.

10. What is callback method?

A callback is a function passed to another function so that it can be called later, for example after an operation completes or for each item being processed. A callback is a function pattern, not a specific JavaScript method.

Example:

```
function greet(name, callback) {
    console.log("Hello " + name);
    callback();
}

greet("Asha", function() {
    console.log("Welcome!");
});
```

Output:

```
Hello Asha
Welcome!
```

The function supplied as the second argument is the callback, and greet calls it after displaying the greeting.